

DG INTERNS HUB

WEEK 5 — VAPT PRACTICAL LEARNING

Advanced Vulnerability Assessment & Penetration Testing

Technical Learning & Practical Report

Prepared by	SAMYUKTHA S.
Internship ID	DG/SEPTEMBER/CYBER/102
Official Assignment Target	http://testphp.vulnweb.com
Authorized Practical Extension	Ubuntu Apache Lab — 192.168.162.131
Primary Tools	Nmap • Nikto • Burp Suite • curl • Apache2

Evidence-led workflow: identify → reproduce → interpret → remediate → retest

Submission note: The report preserves the actual practical evidence and distinguishes the official assignment target from the separately authorized Ubuntu extension.

1. Objective

The objective of this Week 5 practical task is to perform an advanced, evidence-driven Vulnerability Assessment and Penetration Testing workflow using industry-relevant reconnaissance, web-server assessment, HTTP traffic inspection, validation and defensive hardening techniques.

The practical work focuses on understanding how a security analyst moves from initial attack-surface discovery to service identification, web-server enumeration, request interception, finding validation, risk interpretation, remediation and post-remediation retesting.

A central VAPT principle applied throughout the exercise is that a scanner alert is not automatically a confirmed vulnerability. Findings must be reproduced or validated with direct evidence before they are documented as security issues.

Advanced practical objectives

- Identify exposed services and characterize the target attack surface using Nmap.
- Perform web-server enumeration and security-configuration assessment using Nikto.
- Inspect HTTP requests and responses through Burp Suite.
- Validate scanner observations using direct HTTP requests with curl.
- Apply Apache hardening controls to an authorized local target.
- Perform repeatable before/after retesting to demonstrate whether remediation was effective.
- Distinguish target limitations, observations, validated findings and confirmed remediation.

2. Scope, Methodology & Environment

Scope

The official assignment target is **http://testphp.vulnweb.com**. No unauthorized exploitation or modification was performed. Because the official web service could not be reached from the current lab path, the report documents that limitation as evidence.

A separate Ubuntu Apache environment was used as an **authorized practical extension**. Evidence from the Ubuntu target is explicitly separated from the official target and is used to demonstrate the full VAPT-to-hardening workflow.

Methodology

1. Reconnaissance and host/service discovery → 2. Web-server enumeration → 3. HTTP request/response inspection → 4. Finding validation → 5. Risk interpretation → 6. Defensive hardening → 7. Configuration validation → 8. Retesting → 9. Before/after comparison.

Environment

Environment	Observed / Used
Assessment workstation	Kali Linux
Authorized Ubuntu target	192.168.162.131/24
HTTP service	Apache HTTP Server on TCP/80
Other exposed service	OpenSSH on TCP/22

Tool usage

Tool	Advanced use in this assessment
Nmap	SYN scanning, service/version detection, advanced OS/network characterization and full TCP enumeration.
Nikto	Web-server fingerprinting, HTTP-method observation and security-header/configuration checks.
Burp Suite	Proxy interception and inspection of raw HTTP request/response traffic.
curl	Repeatable validation of HTTP headers, ETag behavior and HTTP method restrictions.
Apache2	Authorized local server used for remediation and retesting.

3. Official Target — testphp.vulnweb.com

The official target was assessed first, as required by the assignment. DNS resolution succeeded and the target resolved to **44.228.249.3**. However, TCP connectivity to the web service did not complete from the current lab path.

Important interpretation: this is a connectivity limitation, not a statement that the official website has no vulnerabilities. Because the service could not be reached end-to-end, application-level vulnerability validation could not be completed.

Nmap assessment

The SYN scan, service/version scan, advanced scan and full TCP scan all showed filtered/no-response behavior from the current path. The full TCP scan observed all 65,535 TCP ports as filtered. Because service connections could not be established, no service/version vulnerabilities were claimed.

```
nmap --version
nmap -sS testphp.vulnweb.com
nmap -sV testphp.vulnweb.com
nmap -A testphp.vulnweb.com
nmap -T4 -p- --reason testphp.vulnweb.com
```

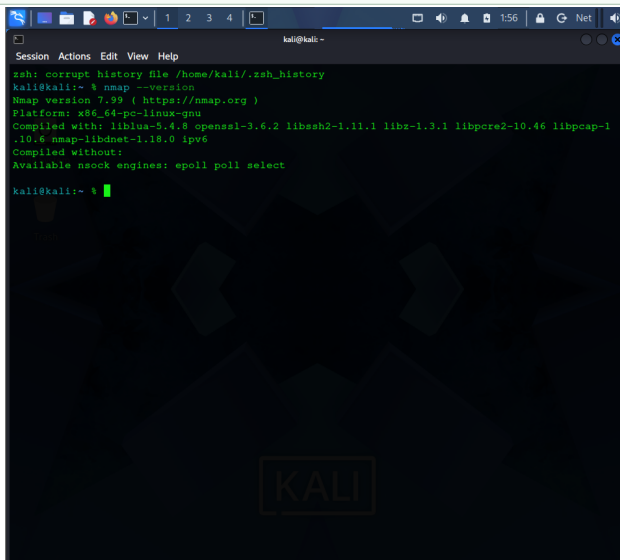


Figure 1. Nmap version information from the assessment workstation.

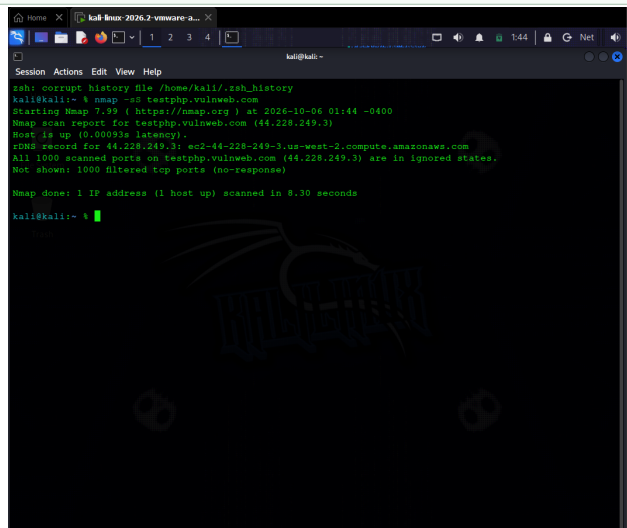


Figure 2. Nmap SYN scan showing the official target's filtered-port response.

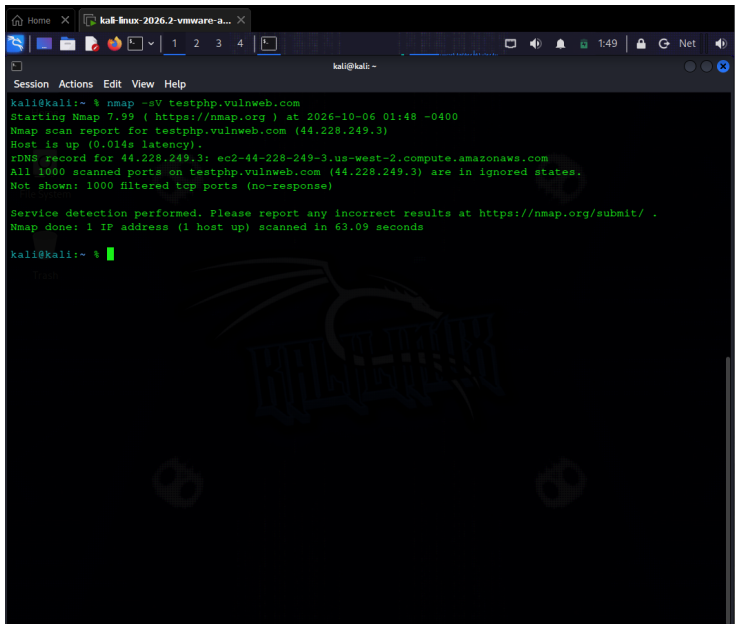


Figure 3. Nmap service/version scan of the official target showing no responsive service ports from the current path.

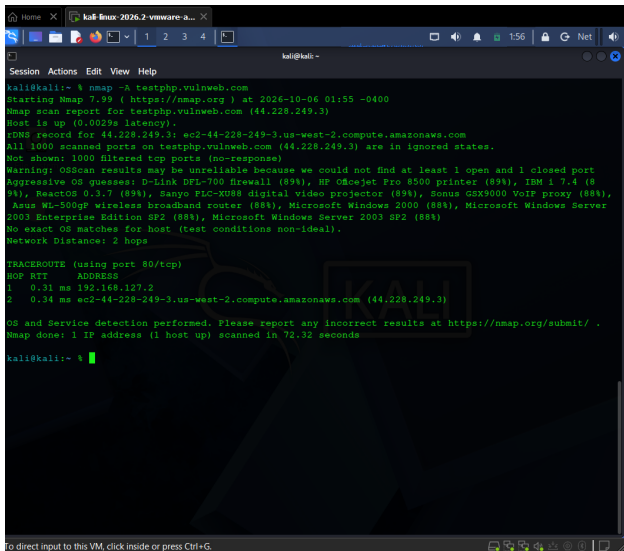


Figure 4. Nmap -A advanced assessment showing filtered ports and unreliable OS inference.

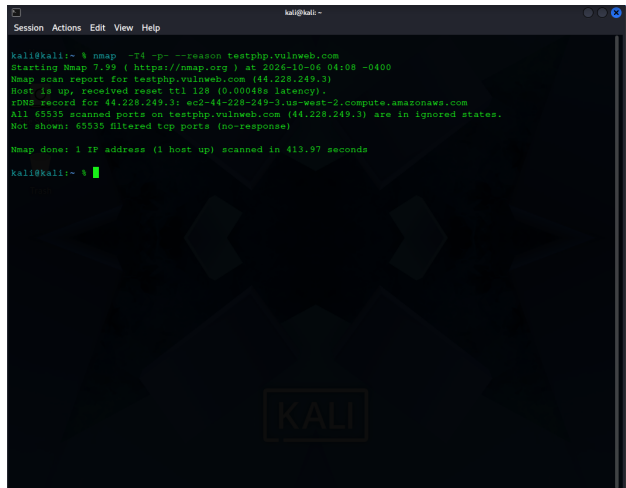


Figure 5. Full TCP enumeration showing all 65,535 ports filtered/no-response.

3. Official Target — Web Tool Connectivity

Nikto assessment

Nikto reported an inability to connect to testphp.vulnweb.com:80. This was treated as a connectivity limitation, not a vulnerability.

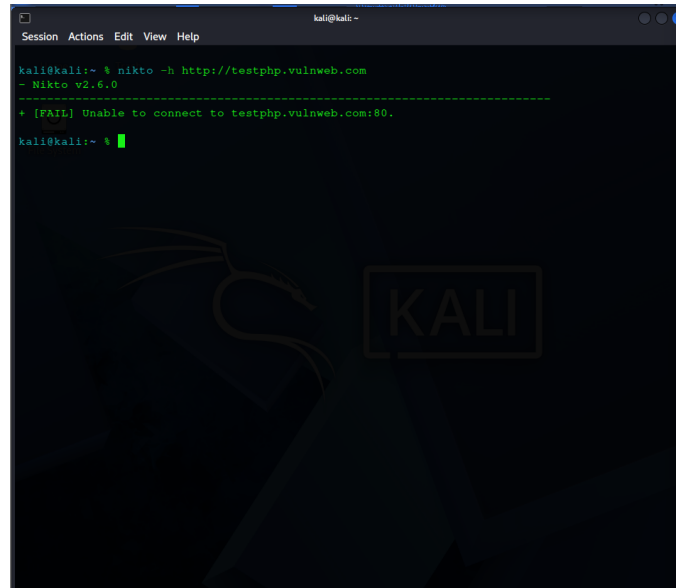


Figure 6. Nikto retry against the official target showing the connection failure.

Burp Suite assessment

The GET request was successfully intercepted, proving that the browser-to-Burp request path was working. Forwarding the request then resulted in a connection failure to the target's port 80. Therefore, proxy capture is evidence of request interception, not evidence that the application was reachable end-to-end.

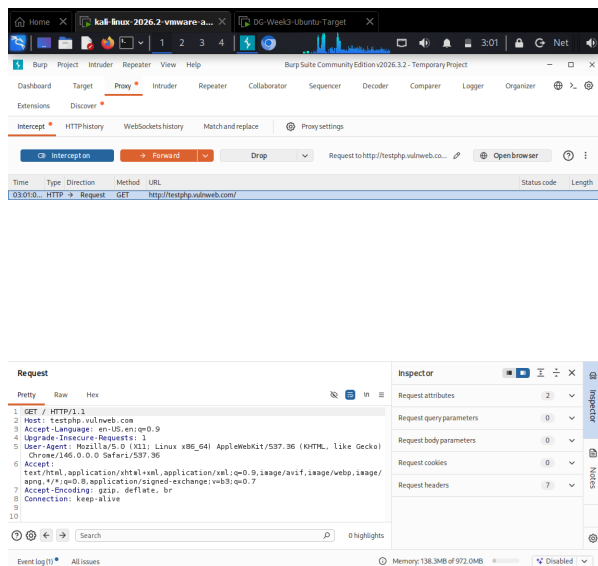


Figure 7. Burp Suite captured the official-target GET request before forwarding.

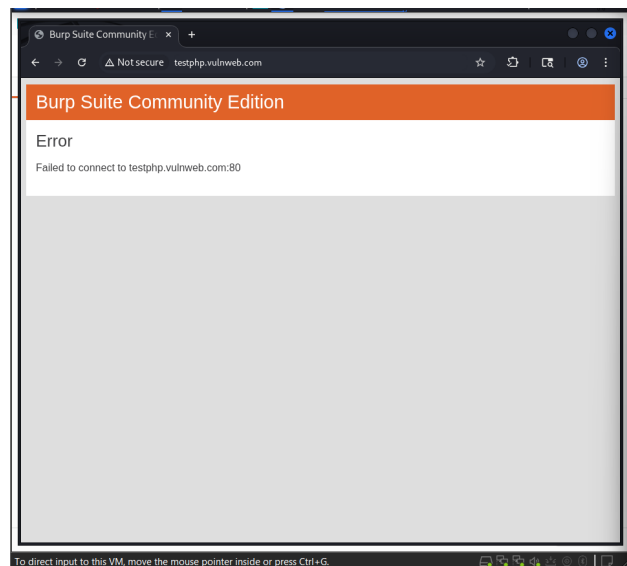


Figure 8. Burp Suite connection failure after forwarding the intercepted request.

Additional web-tool troubleshooting

ZAP was also used during troubleshooting to verify whether the inability to connect was specific to a single application. ZAP showed the target configured for an automated scan while the target remained unreachable. ZAP is retained only as troubleshooting evidence and is not treated as a replacement for the required Burp Suite task.

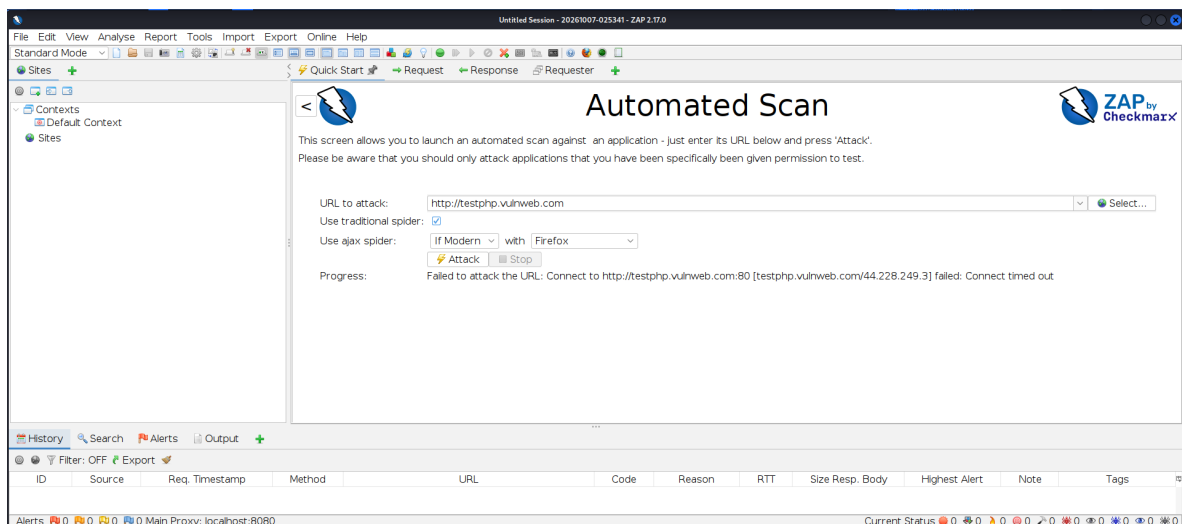


Figure 9. ZAP interface showing the official target remaining unreachable during troubleshooting.

Professional VAPT interpretation: a failed test is not a negative security result. A timeout means the test could not be completed; it does not mean the target is secure.

4. Authorized Ubuntu Lab — Advanced Assessment

The Ubuntu lab was introduced as a separate practical extension so the complete assessment and remediation workflow could be performed against a system under the user's control.

Baseline web service

```
valid_lft forever preferred_lft forever
targetadmin@ubuntu-target:~$ sudo systemctl status apache2
[sudo: authenticate] Password:
• apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-10-07 05:25:04 UTC; 8min ago
   Invocation: 7db8831b91694154a77d86af5f80fb3f
     Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 1672 (apache2)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
     Tasks: 55 (limit: 1827)
    Memory: 8M (peak: 8.3M)
       CPU: 151ms
    CGroup: /system.slice/apache2.service
           └─1672 /usr/sbin/apache2 -k start -DFOREGROUND
             └─1704 /usr/sbin/apache2 -k start -DFOREGROUND
               └─1705 /usr/sbin/apache2 -k start -DFOREGROUND

Oct 07 05:25:04 ubuntu-target systemd[1]: Starting apache2.service - The Apache HTTP Server...
Oct 07 05:25:04 ubuntu-target apachectl[1672]: AH00558: apache2: Could not reliably determine the server's fully qualif
Oct 07 05:25:04 ubuntu-target systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-18/18 (END)
```

Figure 10. Apache service status confirming the local HTTP service was active on the authorized Ubuntu target.



Figure 11. Apache default page response confirming HTTP service availability at 192.168.162.131.

Nmap baseline

The local SYN scan identified TCP/22 as open SSH and TCP/80 as open HTTP. Service/version detection identified OpenSSH 10.2p1 Ubuntu 2ubuntu3.5 and Apache httpd 2.4.66 (Ubuntu).

```

kali@kali:~$ nmap -sS 192.168.162.131
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-07 01:45 -0400
Nmap scan report for 192.168.162.131
Host is up (0.00072s latency).
Not shown: 998 filtered tcp ports (no-response)
22/tcp open  ssh
80/tcp open  http
MAC Address: 00:0C:29:76:16:1D (VMware)

Nmap done: 1 IP address (1 host up) scanned in 7.52 seconds
kali@kali:~$
  
```

Figure 12. Nmap SYN scan of the authorized Ubuntu target.

```

kali@kali:~$ nmap -T4 -sV 192.168.162.131
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-07 01:47 -0400
Nmap scan report for 192.168.162.131
Host is up (0.00074s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.2p1 Ubuntu 2ubuntu3.5 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.66 ((Ubuntu))
MAC Address: 00:0C:29:76:16:1D (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 13.37 seconds
kali@kali:~$
  
```

Figure 13. Nmap service/version detection identifying OpenSSH and Apache.

Commands evidenced in the practical

```

nmap -T4 -sS 192.168.162.131
nmap -T4 -sV 192.168.162.131
  
```

Nikto baseline

Nikto identified Apache/2.4.66 (Ubuntu), an ETag-related information-disclosure observation, missing recommended security headers, and the HTTP methods GET, POST, OPTIONS and HEAD. These observations were not accepted as final findings until direct validation was performed.

```

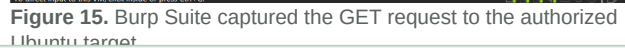
kali@kali:~$ nikto -h http://192.168.162.131
- Nikto v2.6.0

+-----+
+ Target IP:      192.168.162.131
+ Target Hostname: 192.168.162.131
+ Target Port:    80
+ Platform:       Linux/Unix
+ Start Time:     2026-10-07 03:11:52 (GMT-4)
+-----+

+ Server: Apache/2.4.66 (Ubuntu)
+ ERROR: Failed to check for updates: 403
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [999984] /: Server may leak inodes via ETags, header found with file /, inode: 29b0, size: 65c211720Ha0, mtime: gzip. See: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [999990] OPTIONS: Allowed HTTP Methods: GET, POST, OPTIONS, HEAD .
^C

kali@kali:~$
  
```

Figure 14. Nikto baseline scan of the authorized Ubuntu Apache service.



5. Validated Findings

Finding 1 — ETag Information Disclosure

Evidence: Baseline HTTP response contained ETag: "29b0-65c211720ffa0".

Validation: Direct curl -I request reproduced the ETag.

Interpretation: The response exposed resource metadata that was not necessary for this simple lab service. The evidence supports an information-disclosure/configuration observation; it does not by itself prove exploitability of the historical CVE mentioned by Nikto.

```
kali@kali:~$ curl -I http://192.168.162.131 | grep -i ETag
% Total    % Received % Xferd  Average Speed   Time    Time     Current
                                 Dload  Upload   Total   Spent    Left     Speed
0   10672    0     0  0      0     0      0      0  0  0  0  0
ETag: "29b0-65c211720ffa0"
```

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is fully documented in [/usr/share/doc/apache2/README.Debian.gz](#). Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the manual if the apache2-doc package was installed on this server.

The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

```
/etc/apache2/
-- apache2.conf
-- ports.conf
-- mods-enabled
|-- *.load
|-- *.conf
-- conf-enabled
-- *.conf
-- sites-enabled
-- *.conf
```

- `apache2.conf` is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
- `ports.conf` is always included from the main configuration file. It is used to determine the listening ports for incoming connections, and this file can be customized anytime.

Figure 17. Direct curl validation reproducing the ETag header.

Finding 2 — Missing Security Headers

Evidence: Baseline response lacked Content-Security-Policy, Referrer-Policy, X-Content-Type-Options and Permissions-Policy.

Validation: Direct HTTP response inspection confirmed the headers were absent before remediation.

Interpretation: Missing defensive headers reduce browser-side security controls. The severity depends on application behavior and deployment context.

Session Actions Edit View Help

```
kali@kali:~$ curl -I http://192.168.162.131
HTTP/1.1 200 OK
Date: Wed, 07 Oct 2026 07:27:39 GMT
Server: Apache/2.4.66 (Ubuntu)
Last-Modified: Wed, 23 Sep 2026 07:04:11 GMT
ETag: "29b0-65c211720ffa0"
Accept-Ranges: bytes
Content-Length: 10672
Vary: Accept-Encoding
Content-Type: text/html
```

Apache2 Default Page

ubuntu

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache2 HTTP server installed at this site is working properly. You should replace this file (located at `/usr/share/html2/Default`, `00001`) before attempting to operate your HTTP server.

If you are a technical user of the web site and don't know what this page is about, this probably means that the site is currently unavailable due to misconfiguration. If the problem persists, please contact the site's administrator.

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is fully documented in [/usr/share/doc/apache2/README.Debian.gz](#). Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the manual if the apache2-doc package was installed on this server.

The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

```
/etc/apache2/
-- apache2.conf
-- ports.conf
-- mods-enabled
|-- *.load
|-- *.conf
-- conf-enabled
-- *.conf
-- sites-enabled
-- *.conf
```

- `apache2.conf` is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
- `ports.conf` is always included from the main configuration file. It is used to determine the listening ports for incoming connections, and this file can be customized anytime.

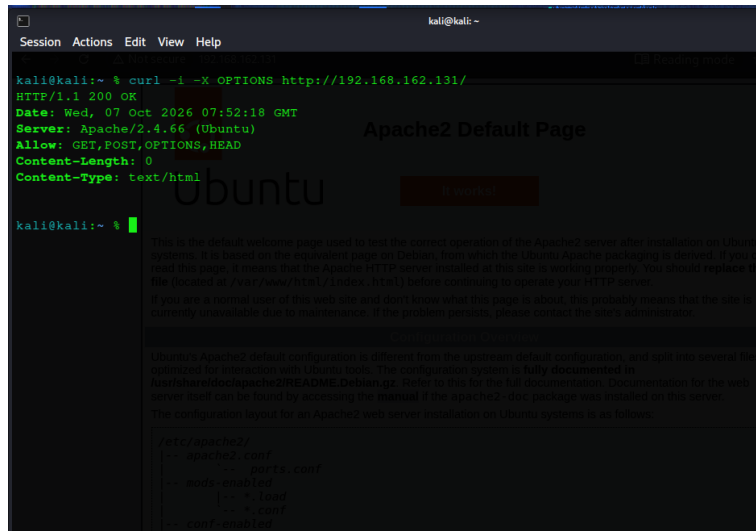
Figure 18. Baseline HTTP response showing the security-header gap.

Finding 3 — Unnecessary HTTP Methods

Evidence: Baseline OPTIONS response returned HTTP 200 OK and Allow: GET, POST, OPTIONS, HEAD.

Validation: Direct OPTIONS request reproduced the behavior.

Interpretation: The methods were not automatically treated as exploitable vulnerabilities. The hardening objective was to reduce the allowed method set for the simple static lab service.



```
kali@kali:~$ curl -i -X OPTIONS http://192.168.162.131/
HTTP/1.1 200 OK
Date: Wed, 07 Oct 2026 07:52:18 GMT
Server: Apache/2.4.66 (Ubuntu)
Allow: GET, POST, OPTIONS, HEAD
Content-Length: 0
Content-Type: text/html

kali@kali:~$
```

Figure 19. Direct OPTIONS validation showing the baseline Allow header.

Finding discipline

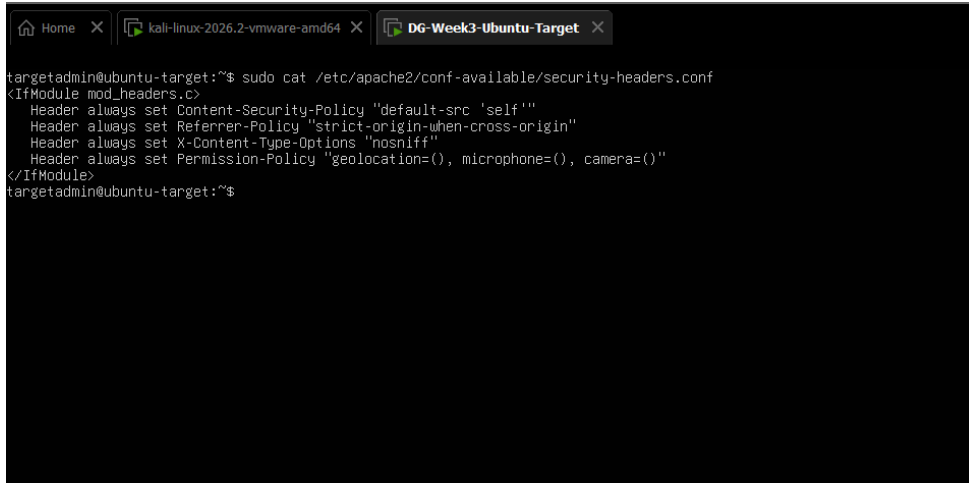
HSTS was not included as a validated finding because the local assessment was performed over HTTP and no HTTPS deployment was established. This avoids overstating scanner recommendations as confirmed vulnerabilities.

6. Defense & Hardening

The remediation stage was performed only against the authorized Ubuntu environment. The practical evidence shows the configuration was changed, syntax-tested, reloaded, and then verified from the assessment workstation.

Security-header hardening

Apache's headers module was enabled and security-response headers were configured for the local web service. The post-hardening response showed the expected headers in the HTTP response.



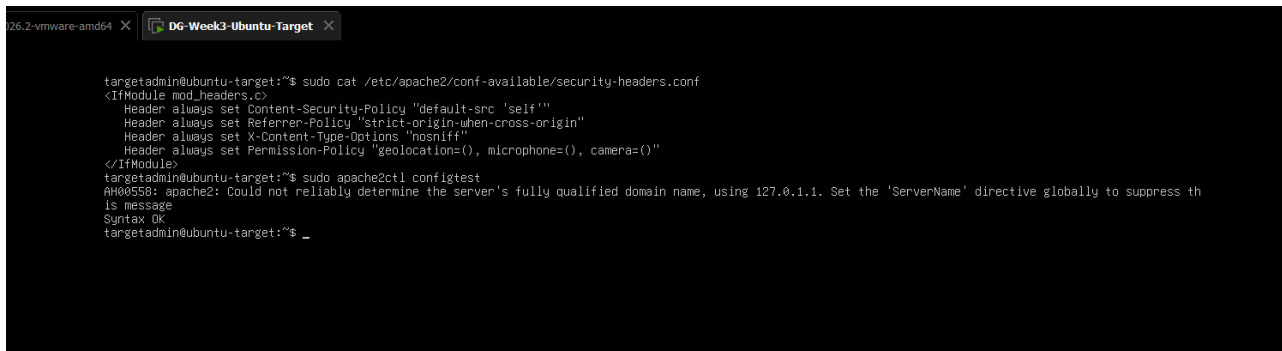
```
targetadmin@ubuntu-target:~$ sudo cat /etc/apache2/conf-available/security-headers.conf
<IfModule mod_headers.c>
  Header always set Content-Security-Policy "default-src 'self'"
  Header always set Referrer-Policy "strict-origin-when-cross-origin"
  Header always set X-Content-Type-Options "nosniff"
  Header always set Permission-Policy "geolocation=(), microphone=(), camera=()"
</IfModule>
targetadmin@ubuntu-target:~$
```

Figure 20. Apache security-header configuration used for the authorized local service.

```
<IfModule mod_headers.c>
Header always set Content-Security-Policy "default-src 'self'"
Header always set Referrer-Policy "strict-origin-when-cross-origin"
Header always set X-Content-Type-Options "nosniff"
Header always set Permission-Policy "geolocation=(), microphone=(), camera=()"
</IfModule>
```

Apache configuration validation

Apache configuration syntax was tested before the service was reloaded. The supplied evidence shows **Syntax OK**.



```
targetadmin@ubuntu-target:~$ sudo cat /etc/apache2/conf-available/security-headers.conf
<IfModule mod_headers.c>
  Header always set Content-Security-Policy "default-src 'self'"
  Header always set Referrer-Policy "strict-origin-when-cross-origin"
  Header always set X-Content-Type-Options "nosniff"
  Header always set Permission-Policy "geolocation=(), microphone=(), camera=()"
</IfModule>
targetadmin@ubuntu-target:~$ sudo apache2ctl configtest
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress th
is message
Syntax OK
targetadmin@ubuntu-target:~$ _
```

Figure 21. Apache configuration test showing Syntax OK before reload.

Advanced Vulnerability Assessment & Penetration Testing

Advanced Vulnerability Assessment & Penetration Testing



Advanced Vulnerability Assessment & Penetration Testing

Advanced Vulnerability Assessment & Penetration Testing

Advanced Vulnerability Assessment & Penetration Testing

Advanced Vulnerability Assessment & Penetration Testing



Advanced Vulnerability Assessment & Penetration Testing



Advanced Vulnerability Assessment & Penetration Testing

HTTP method hardening

The initial method-restriction configuration was corrected after Apache reported a syntax error. The final rule was placed inside the appropriate **<Directory /var/www/html>** container with **<LimitExcept GET HEAD>**. The corrected configuration passed Syntax OK and Apache remained active after reload.

```

GNU nano 8.7.1 /etc/apache2/conf-available/http-methods-harder
<LimitExcept GET HEAD>
    Require all denied
</LimitExcept>

```

Figure 25. Corrected HTTP-method hardening rule using LimitExcept GET HEAD.

```

<LimitExcept GET HEAD>
Require all denied
</LimitExcept>

```

```

targetadmin@ubuntu-target:~$ sudo apache2ctl configtest
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress th
is message
Syntax OK
targetadmin@ubuntu-target:~$ sudo systemctl reload apache2
targetadmin@ubuntu-target:~$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-10-07 08:12:11 UTC; 45min ago
   Invocation: 1abff2dca48c480a1767e6484b26c02
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 33666 ExecReload=/usr/sbin/apachectl graceful (code=exited, status=0/SUCCESS)
  Main PID: 33687 (apache2)
    Status: "Total requests: 1; Idle/Busy workers 100/0;Requests/sec: 0.000364; Bytes served/sec:  0 B/sec"
   Tasks: 55 (limit: 1027)
  Memory: 5.7M (peak: 8.4M)
     CPU: 491ms
   CGroup: /system.slice/apache2.service
           └─33687 /usr/sbin/apache2 -k start -DFOREGROUND
             33670 /usr/sbin/apache2 -k start -DFOREGROUND
             33671 /usr/sbin/apache2 -k start -DFOREGROUND

Oct 07 08:40:29 ubuntu-target apachectl[33427]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress th
is message
Oct 07 08:40:29 ubuntu-target systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
Oct 07 08:49:57 ubuntu-target systemd[1]: Reloading apache2.service - The Apache HTTP Server...
Oct 07 08:49:57 ubuntu-target apachectl[33566]: AH00528: Syntax error on line 2 of /etc/apache2/conf-enabled/http-methods-hardening.conf:
Oct 07 08:49:57 ubuntu-target apachectl[33566]: <LimitExcept not allowed here
Oct 07 08:49:57 ubuntu-target systemd[1]: apache2.service: Control process exited, code=exited, status=1/FAILURE
Oct 07 08:49:57 ubuntu-target systemd[1]: Reload failed for apache2.service - The Apache HTTP Server.
Oct 07 08:57:56 ubuntu-target systemd[1]: Reloading apache2.service - The Apache HTTP Server...
Oct 07 08:57:56 ubuntu-target apachectl[33666]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress th
is message
Oct 07 08:57:56 ubuntu-target systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.
targetadmin@ubuntu-target:~$

```

Figure 26. Apache configuration validation and reload/status evidence for the HTTP-method restriction.

Practical learning point: configuration changes must be syntax-validated before reloads, and a failed reload must be investigated from the actual Apache error rather than guessed.

7. Retesting & Before/After Validation

The same validation requests were repeated after hardening. This creates an objective before/after comparison rather than relying only on the presence of a changed configuration file.

Control	Before	After	Outcome
ETag	ETag: "29b0-65c211720ffa0"	ETag absent	Hardened
Security headers	CSP / Referrer-Policy / X-Content-Type-Options / Permissions-Policy absent	All four present	Hardened
HTTP methods	OPTIONS → 200 OK; GET, POST, OPTIONS, HEAD	OPTIONS → 403 Forbidden	Restricted

```

kali@kali:~$ curl -I http://192.168.162.131/
HTTP/1.1 200 OK
Date: Wed, 07 Oct 2026 08:42:12 GMT
Server: Apache/2.4.66 (Ubuntu)
Content-Security-Policy: default-src 'self'
Referrer-Policy: strict-origin-when-cross-origin
X-Content-Type-Options: nosniff
Permission-Policy: geolocation=(), microphone=(), camera=()
Last-Modified: Wed, 23 Sep 2026 07:04:11 GMT
Accept-Ranges: bytes
Content-Length: 10672
Vary: Accept-Encoding
Content-Type: text/html; charset=iso-8859-1

This is the default welcome page used to test the correct operation of the Apache2 server after
installation on Ubuntu. This page is located at /usr/share/html/index.html. If you are a
normal user of this web site and don't know what this page is about, this probably means the site is
unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and you may
need to adjust the configuration for your system. The configuration system is fully documented in
/usr/share/doc/apache2/README.Debian.gz. Refer to this file for the full documentation. Once
server hard can be found by accessing the manual if the apache2-doc package was installed on this server.
The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|   |-- mods-enabled
|   |   |-- *.load
|   |   |-- *.conf
|   |-- conf-enabled
|   |   |-- *.conf
|   |-- sites-enabled
|   |   |-- *.conf
|   |-- *.conf

- apache2.conf is the main configuration file. It puts the pieces together by including all remaining configuration
  files when starting up the web server.
- ports.conf is always included from the main configuration file. It is used to determine the listening ports for
  incoming connections, and this file can be customized anytime.
  
```

Figure 27. Post-hardening HTTP response showing the ETag header absent and the configured security headers present.

```

kali@kali:~$ curl -i -X OPTIONS http://192.168.162.131/
HTTP/1.1 403 Forbidden
Date: Wed, 07 Oct 2026 08:59:37 GMT
Server: Apache/2.4.66 (Ubuntu)
Content-Security-Policy: default-src 'self'
Referrer-Policy: strict-origin-when-cross-origin
X-Content-Type-Options: nosniff
Permission-Policy: geolocation=(), microphone=(), camera=()
Content-Length: 320
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<hr>
<address>Apache/2.4.66 (Ubuntu) Server at 192.168.162.131 Port 80</address>
</body></html>

This is the default welcome page used to test the correct operation of the Apache2 server after
installation on Ubuntu. This page is located at /usr/share/html/index.html. If you are a
normal user of this web site and don't know what this page is about, this probably means the site is
unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and you may
need to adjust the configuration for your system. The configuration system is fully documented in
/usr/share/doc/apache2/README.Debian.gz. Refer to this file for the full documentation. Once
server hard can be found by accessing the manual if the apache2-doc package was installed on this server.
The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|   |-- mods-enabled
|   |   |-- *.load
|   |   |-- *.conf
|   |-- conf-enabled
|   |   |-- *.conf
|   |-- sites-enabled
|   |   |-- *.conf
  
```

Figure 28. Post-hardening OPTIONS request returning HTTP 403 Forbidden.

8. Evidence & Practical Learning

The supplied evidence set contains the official-target Nmap scans, Nikto connection-failure evidence, Burp official-target request/failure evidence, Ubuntu Nmap evidence, Ubuntu Nikto scan, Ubuntu Burp request/response, baseline HTTP headers, ETag validation, hardening configuration/status screenshots and post-hardening retests.

Workflow stage	Evidence used	Security purpose
Nmap	Official-target scans + Ubuntu SYN/service scans	Attack-surface and service characterization
Nikto	Official-target connection failure + Ubuntu baseline scan	Web-server enumeration and candidate observations
Burp Suite	Official-target GET/failure + Ubuntu GET/response	Direct HTTP traffic visibility
Validation	curl ETag, header and OPTIONS requests	Convert scanner observations into reproducible evidence
Defense	Apache header, ETag and method configurations	Apply targeted hardening
Retest	Post-hardening ETag, headers and OPTIONS responses	Verify remediation from the client side

Key learnings

- Reconnaissance results must be interpreted in network context.
- Scanner output requires validation before being documented as a finding.
- Burp Suite provides visibility into the exact HTTP request/response flow.
- curl is useful for repeatable, low-noise validation of web-server behavior.
- Apache hardening should be syntax-tested before reload and then retested from the client side.
- Before/after testing provides stronger evidence than a configuration change alone.

Advanced analyst perspective

The most important outcome of this exercise is the transition from tool output to defensible security evidence: identify → reproduce → interpret → remediate → retest. This is the core workflow that separates a basic scan from a practical VAPT assessment.

9. Conclusion

The Week 5 exercise demonstrated an end-to-end VAPT workflow using Nmap, Nikto and Burp Suite, supported by curl-based validation and Apache hardening in an authorized Ubuntu environment.

The official target, **testphp.vulnweb.com**, was assessed as required. Its network/web connectivity limitation was documented honestly, and no unsupported vulnerability claims were made. The report does not infer that the official target is secure.

The separate Ubuntu lab enabled deeper hands-on work: Nmap identified the exposed SSH and HTTP services; Nikto produced actionable web-server observations; Burp Suite confirmed HTTP request/response behavior; curl validated the findings; Apache was hardened; and the same tests were repeated to verify the remediation.

Final technical status

Area	Status
Official target assessment	Completed as far as connectivity permitted
Ubuntu VAPT extension	Completed
Finding validation	Completed
Defense / hardening	Completed
Retesting	Completed
Before/after evidence	Completed

Evidence completeness note

The final report uses the actual screenshots supplied in the evidence ZIP and places them beside the workflow stage where each item was generated. No placeholder or fabricated screenshot has been substituted.

Report basis

This polished edition is an evidence-preserving presentation revision of the supplied Week 5 report. Its technical claims, workflow and observations are grounded in the supplied report and evidence archive.